

▮ CS50 Scratch Overview

Brief Overview

This note covers **Scratch programming** and was created from the [CS50x 2026 - Lecture 0 - Scratch](#) YouTube video. The content covers AI as a programming tool, VS Code workflow, Scratch fundamentals, game creation, and algorithmic efficiency.

Key Points

- Learn how **AI tools** can act as a coding co-pilot and build a Python chatbot.
 - Master VS Code basics and Scratch block programming to create interactive stories and games.
 - Understand algorithmic concepts like binary search and game design patterns such as abstraction and modularity.
-

▮ AI as a Programming Tool

Artificial Intelligence (AI) – Systems that can process natural-language prompts, generate code, locate bugs, and suggest new features.

- AI helps **solve problems faster** by acting as a co-pilot rather than a replacement.
- Even with AI assistance, **understanding fundamentals** remains essential; AI extends, not eliminates, core knowledge.

▮ Development Environment: VS Code

Visual Studio Code (VS Code) – A free, open-source text editor used widely in industry for writing and running code.

- Integrated **terminal** allows issuing commands directly from the editor.
- Enables rapid experimentation with scripts such as a Python chatbot.

▮ Building a Simple Chatbot (Python)

Chatbot – A program that sends a user's prompt to an AI model via an API and returns the model's response.

```
import openai

client = openai.OpenAI() # Create a client for the OpenAI API
prompt = input("Enter your question: ") # Get user input
response = client.chat.completions.create(
    model="gpt-5",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)
```

- **Static version**: hard-coded question (e.g., “What is CS 50?”).
- **Dynamic version**: uses input() to accept any user query.
- **System prompt** can be added to constrain the model (e.g., “limit answer to one sentence”).
- Personality tweaks (e.g., “pretend you’re a cat”) are set via the system prompt.

▢ Rubber Duck Debugging

Rubber Duck Debugging – Explaining code problems aloud to an inanimate object to clarify thinking and uncover logical errors.

- CS 50 provides a **virtual rubber duck** that acts like a human tutor: guides without giving outright answers.
- Encouraged alongside AI tools for problem-solving.

▢ Foundations of Computer Science

Computational Thinking – Applying CS concepts (algorithms, data structures, abstraction) to solve real-world problems.

- **Input → Black Box → Output** model illustrates problem solving.
- Programming languages (Scratch, C, Python, SQL, HTML/CSS/JS) are tools for implementing solutions.

▢ Number Systems

System	Base	Digits	Typical Use
Unary	1	0 (single symbol)	Counting with fingers; very limited.

Binary	2	0, 1	Core of digital computers; maps to on/off electricity.
Decimal	10	0-9	Everyday human arithmetic.

□ Unary (Base-1)

Unary notation – Represents a quantity by repeated instances of a single symbol (e.g., tally marks).

- Simple but **inefficient** for larger numbers; requires many symbols.

□ Binary (Base-2)

Bit – The smallest unit of data, representing either 0 (off) or 1 (on).

- **Byte** = 8 bits.
- Binary patterns correspond to powers of two: 2^0 , 2^1 , 2^2 , ...
- Example conversion:

Binary	Decimal
000	0
001	1
010	2
011	3
100	4
111	7
1000	8

- Adding bits expands the representable range (2^n values for n bits).

□ Decimal (Base-10)

Place value – Each digit's weight is a power of ten: 10^0 , 10^1 , 10^2 , ...

- Number 123 = $1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0$.

□ Data Units

- **Bit** – Single binary digit (0 or 1).
- **Byte** – 8 bits; foundational unit for larger measures.
- **Kilobyte (KB)** – 10^3 bytes (or 2^{10} bytes in binary contexts).
- **Megabyte (MB)** – 10^6 bytes; **Gigabyte (GB)** – 10^9 bytes; **Terabyte (TB)** – 10^{12} bytes.

These units describe storage capacity and memory size, reflecting how computers aggregate bits into usable quantities.

□ Bit Depth & Value Ranges

Bit depth – The number of bits used to represent a single value.

- An **8-bit** pattern can express $2^8 = 256$ distinct values (0–255).
- Adding a ninth bit doubles the range: $2^9 = 512$, and so on.
- **32-bit** integers provide $2^{32} \approx 4$ billion possible values.
 - When signed, the positive range is up to $2^{31} - 1 \approx 2$ billion .
- Modern systems often use **64-bit** values, giving $2^{64} \approx 1.84 \times 10^{19}$ possible patterns.

These ranges are why early graphics hardware was limited to **256 colors** (8bits per pixel) and why modern devices can handle far larger palettes.

□ Character Encoding

Encoding – A standardized mapping from numeric values (bits) to symbols such as letters, digits, or control characters.

ASCII (American Standard Code for Information Interchange)

- Uses **7 bits** (historically) but stored in a full **8-bit** byte.
- Example mapping:

Decimal	Hex	Binary	Symbol
65	41	01000001	A
66	42	01000010	B
67	43	01000011	C
...	...	...	...

- **Lowercase** letters start at 97; the offset between upper- and lower-case is 32 (e.g., $97 - 65 = 32$). Changing the 32-place bit flips case.

Reading Bytes as Text

- A three-byte sequence 72, 73, 33 corresponds to **“HI!”** (ASCII 72 = ‘H’, 73 = ‘I’, 33 = ‘!’).
 - The same bytes could be interpreted differently in another program (e.g., as color values).
-

Unicode & Emoji

Unicode – A superset of character encodings that assigns a unique code point to virtually every written symbol, emoji, and historic script.

- Extends beyond 8 bits: common forms use **16-bit**, **24-bit**, or **32-bit** code points.
 - With **32 bits**, up to $2^{32} = 4,294,967,296$ distinct characters are possible.
 - Emoji are just Unicode characters; different platforms render them with their own fonts (e.g., Apple, Google, Microsoft).
-

Color Representation (RGB)

RGB – A color model that combines red, green, and blue light intensities to produce a wide spectrum of colors.

- Each channel is typically an **8-bit** value (0–255).
- A **24-bit** color = 8 bits (red) + 8 bits (green) + 8 bits (blue).

Example	Red	Green	Blue	Appearance
72, 23, 33	72	23	33	Dark yellow (approx.)
0, 0, 0	0	0	0	Black (absence of light)
255, 255, 255	255	255	255	White (full intensity)

- Hexadecimal notation (e.g., #00FF00) is a compact way to write these three bytes.
-

Pixels, Images & File Sizes

Pixel – The smallest addressable element in a raster image, storing color via an RGB triple.

- One pixel = **3 bytes** (24 bits) in standard RGB images.
 - An image of 1000×1000 pixels $\rightarrow 1,000,000 \times 3 \text{ bytes} \approx \mathbf{3 \text{ MB}}$.
 - Larger images or video frames quickly reach **megabyte** or **gigabyte** sizes because each frame multiplies the pixel count.
-

□ Video as Frame Sequences

Video – A rapid series of still images (frames) displayed sequentially to create the illusion of motion.

- Typical playback: **30 frames/second** (fps).
 - Each frame is essentially an image; storing many frames dramatically increases data size.
-

□ Audio Encoding Basics

Audio sample – A numeric representation of sound at a specific moment.

Key parameters for a simple digital note:

1. **Frequency (pitch)** – Determines the tone (e.g., 440 Hz = A₄).
 2. **Duration** – How long the note is held.
 3. **Amplitude (loudness)** – Volume of the sound.
- By encoding these three values (often as 8-bit or 16-bit numbers), a digital audio file can be reconstructed by a player that knows the interpretation scheme.
-

□ Interpreting Binary in Context

Contextual interpretation – The same sequence of bits can represent different data types depending on how a program reads them.

- The byte 65 may be displayed as the character **'A'** in a text editor, as the integer **65** in a calculator, or as part of an RGB triple in an image.
- The programmer's role is to **declare** the intended meaning (number, letter, color, etc.) so the computer can render the correct representation.

□ Algorithms & Efficiency

Algorithm – A step-by-step set of instructions that solves a specific problem.

- **Linear search** (scan from the first entry to the last)
 - Works for any ordered list but requires up to **n** steps for n items.
 - Time grows proportionally to the size of the input → **$O(n)$** .
- **Binary search** (divide-and-conquer)
 - Requires the list to be sorted.
 - Repeatedly inspect the middle element, discard the half that cannot contain the target, and continue on the remaining half.
 - Number of steps $\approx \log_2 n \rightarrow$ **$O(\log n)$** , which grows much more slowly than linear time.
- **Efficiency comparison**

Algorithm	Steps (worst case)	Growth rate
Linear search	n	Linear ($O(n)$)
“Every-other” scan	$n/2 + 1$	Still linear
Binary search	$\lceil \log_2 n \rceil + 1$	Logarithmic ($O(\log n)$)

- **Why logarithmic matters:**
 - Doubling the data set only adds **one** extra step in a binary search, whereas linear search would double the total steps.
 - Example: a 1 000-page phone book → ~10 steps with binary search; a 2 000-page book → ~11 steps.
- **Practical impact:**
 - Modern contact apps, databases, and search features rely on logarithmic-time algorithms to remain fast even with thousands or millions of entries.

□ Pseudocode & Control Structures

Pseudocode – Human-readable description of an algorithm using plain language and minimal syntax.

Typical building blocks

Block type	Symbolic name	Purpose
------------	---------------	---------

Function	verb/action	Performs a specific task (e.g., searchContact).
Conditional	<i>if / else</i>	Chooses a path based on a Boolean expression (true/false).
Loop	<i>while / for</i>	Repeats a set of steps until a condition changes.
Boolean expression	<i>question</i>	Evaluates to true or false (e.g., target ≤ middle).

Sample pseudocode for binary search

```
function binarySearch(list, target):
    low ← 0
    high ← length(list) - 1
    while low ≤ high:
        mid ← (low + high) / 2
        if list[mid] = target:
            return mid          // Found
        else if target < list[mid]:
            high ← mid - 1      // Search left half
        else:
            low ← mid + 1       // Search right half
    return notFound
```

- The loop repeats until the **search interval** collapses to a single element; each iteration halves the interval size.
- **Corner case handling** (notFound) ensures the algorithm terminates even when the target is absent, preventing infinite loops or crashes.

⚙️ From Bits to Instructions: Compilers & Abstraction

Abstraction – Hiding lower-level details behind simpler, higher-level constructs.

- **Machine language**: Patterns of zeros and ones that directly trigger operations such as **add**, **load**, **store**, or **print**.

- **Assembly language**: Symbolic mnemonics (e.g., ADD, MOV) that map one-to-one to machine instructions.
 - **Compiler**: A program that translates a high-level language (C, Python, etc.) into machine code, handling the many layers of abstraction automatically.
 - **Why it matters**
 - Developers write *source code* using readable syntax; the compiler performs the heavy lifting of converting it to binary that the CPU executes.
 - This enables rapid development, portability across hardware, and focus on algorithmic logic rather than hardware minutiae.
 - **Historical note**: Early programmers used punch cards or raw binary to encode programs; the invention of compilers made programming accessible to a far larger audience.
-

Scratch: Visual Programming Foundations

Scratch – A block-based programming environment that lets users create interactive stories, games, and animations by dragging and connecting visual code blocks.

Core concepts

- **Blocks palette** – Color-coded categories representing functions, loops, conditionals, events, and more.
- **Sprites** – On-screen objects (e.g., the default cat) that can be moved, scripted, and animated.
- **Stage coordinates** – An XY plane where (0,0) is the center; positions are expressed in pixels (e.g., x = 240, y = -180).

Typical event block

- **when green flag clicked** – Starts the program when the user clicks the green flag (the “go” signal).

Example workflow

1. Drag the **event block** onto the scripting area.
 2. Attach a **motion block** (e.g., move 10 steps) to define what the sprite does after the flag is clicked.
 3. Optionally add a **control block** like repeat 10 to loop the motion.
- This visual assembly mirrors the same structures discussed earlier (functions, loops, conditionals) but in a beginner-friendly format.

Connection to earlier topics

- The **binary search** algorithm can be implemented in Scratch using a combination of **variables** (low, high, mid), **loops**, and **if-else** blocks.
- Scratch’s “code is just an implementation of an algorithm” reinforces the idea that any algorithm can be expressed in many languages, from low-level machine code to high-level visual blocks.

Scratch Programming Basics (Continuation)

Sprite – A movable object on the Scratch stage (the default sprite is a cat).

- Dragging a puzzle piece (block) onto the scripting area **creates a function call**.
- Blocks **snap together** when their shapes match, providing visual “magnetism.”

▮ Blocks, Arguments, and Side Effects

Argument (parameter) – An input value supplied to a block that influences its behavior.

- The **say** block displays a speech bubble (a **side effect**).
- The **ask ... and wait** block prompts the user and **returns a value** that is stored in a variable (e.g., answer).

Block type	Visual cue	Produces
Side-effect block	White oval (editable)	Visible change (speech bubble, sound)
Return-value block	Blue oval (non-editable)	Hidden value stored in a variable

▮ Building Text with Variables

Variable – A named storage location that can hold numbers, strings, or other data.

- After ask, the user’s response is placed in the variable answer.
- The **join** block concatenates two strings (e.g., "Hello, " + answer).

Workflow:

1. ask "What’s your name?" and wait → stores input in answer.
2. join "Hello, " answer → produces "Hello, ".
3. say → displays the combined text.

□ Controlling Timing with Loops and Wait

Loop – Repeats a set of blocks a specified number of times.

- **repeat N** repeats its interior blocks N times.
- **wait 1 sec** pauses execution for a given duration.

Using a loop prevents the need to copy-paste identical blocks (DRY principle).

□ Defining Custom Blocks (Abstraction)

Custom block – A user-created block that encapsulates a sequence of actions.

1. Click **Make a Block** → name it (e.g., meow).
2. Inside the definition, place the desired actions (e.g., play sound meow until done + wait 1 sec).
3. Drag the new meow block wherever needed.

Adding an argument:

- Edit the custom block, add an input called N.
- Inside the definition, wrap the actions with repeat N.

Result: meow 3 plays the meow sound three times with a one-second pause between each.

□ Conditional Logic and Event Handling

Conditional – Executes a block only when a Boolean expression is true.

- **if then** checks a condition each time it is evaluated.
- Example condition: **touching mouse-pointer?** (true when the cursor is over the sprite).

Putting it together:

- **when green flag clicked** → start program.
- **forever** → continuously evaluate inside.
- **if touching mouse-pointer? then** → play a sound (e.g., meow).

The forever loop keeps the check active; without it the program would run once and miss later interactions.

□ Design Principles Illustrated in Scratch

Issue	Poor practice	Better practice
-------	---------------	-----------------

Duplicated code	Copy-pasting the same sound-wait sequence three times	Use a repeat loop
Scattered constants	Changing the wait time in multiple places	Centralize the value in one block (e.g., a custom block argument)
Hard-coded logic	Fixed number of repetitions	Parameterize with an input (N)

These principles mirror standard software engineering concepts such as **modularity**, **reuse**, and **maintainability**.

▮ **Example: “Pet the Cat” Program**

- **Goal:** Play a meow sound whenever the mouse pointer touches the cat sprite.
- **Implementation:**
 1. when green flag clicked
 2. forever
 3. if touching mouse-pointer? then → play sound meow until done

The forever loop ensures the condition is re-evaluated continuously, allowing the cat to react at any moment.

▮ **Example: “Oscar Time” Game Overview**

- **Purpose:** Drag falling trash into a trash-can sprite before a song ends.
- **Key techniques used:**

Technique	Scratch construct
Sprite costumes	Different images for the trash can (lid up/down, Oscar emerging).
Event handling	when green flag clicked to start the game timer and spawn objects.
Collision detection	if touching ? then → increase score, hide trash.
Sound synchronization	play sound until done coordinated with object spawning.
Scoring	Variables (score) updated on each successful catch.

The project demonstrates how the same building blocks (events, loops, conditionals, variables, and sounds) combine to create an interactive application.

These notes capture the core Scratch concepts presented, from basic block mechanics to higher-level design patterns such as abstraction and modularity. Use them as a reference when constructing or analyzing Scratch programs.

□ Oscar Trash-Can Game (Scratch)

Game concept – A sprite representing a trash can (Oscar) reacts to mouse interaction, while separate trash sprites fall from the top of the screen and must be “collected” by the can.

- The **animation** is achieved by swapping costumes on the Oscar sprite (closed lid vs. open lid).
- Multiple **sprites** (Oscar, trash pieces, background objects) cooperate through independent scripts that run in parallel.

□ Animation via Costumes & Loops

Costume animation – Changing a sprite’s costume frame-by-frame to give the illusion of motion.

- Oscar’s script:
 - switch costume to Oscar-1 (closed lid) at start.
 - **Forever loop**:
 - *If touching mouse-pointer?* → switch costume to Oscar-2 (lid open).
 - *Else* → switch costume to Oscar-1.
- This mirrors the **conditional** pattern introduced earlier (section on pseudocode).

□ Positioning, Randomness & Gravity

Random positioning – Selecting an X-coordinate uniformly within a defined range.

- When the green flag is clicked, each trash sprite:
 - go to x = random(0, 240) → confines trash to the right-hand half of the stage.
 - go to y = 180 → places the sprite at the top edge (the “sky”).
- **Gravity simulation**: forever change y by -1 moves the trash downward one pixel each iteration.
- Scratch automatically prevents sprites from leaving the stage, so the falling sprite stops at the bottom without extra code.

□ Refactoring with Custom Blocks (Functions)

Custom block – A user-defined command that encapsulates a reusable sequence of actions.

- Original versions duplicated the “go to top” code in two separate scripts.
- A **custom block** named go to top was created:
 - Inside the block: go to x = random(0, 240) and go to y = 180.
- Both scripts now simply call go to top, eliminating redundancy and adhering to the **DRY (Don’t Repeat Yourself)** principle discussed earlier.

□ Scoring with Variables

Variable – A named storage location that holds a value which can change over time.

- A variable score is initialized to 0 when the game starts.
- In the trash sprite’s script:
 - if touching Oscar → change score by 1.
- Scratch automatically displays the current score on the stage, providing immediate feedback.

□ Parallel Scripts & Event Handling

Parallel scripts – Multiple “when green flag clicked” scripts attached to the same sprite run concurrently.

- Trash sprite example:
 - **Script 1**: Sets drag mode to *draggable* and handles falling motion.
 - **Script 2**: Detects collision with Oscar and teleports the trash back to the top (go to top).
- This parallelism mirrors the **multi-process** idea from the earlier discussion of algorithms running simultaneously.

□ Wall Detection & Keyboard Movement (Harvard Sprite)

Wall detection – Checking whether a sprite touches the left or right edge of the stage and correcting its position.

- At start: go to x = 0, y = 0.
- **Forever loop** monitors keyboard input:
 - up arrow → change y by 1.
 - down arrow → change y by -1.
 - right arrow → change x by 1.

- left arrow → change x by -1.
- Simultaneously, a separate script checks for wall collisions:
 - *If touching left wall* → change x by 1 (step back).
 - *If touching right wall* → change x by -1.

✂ Enemy Behaviors: Yale & MIT Sprites

Autonomous enemy – A sprite that moves on its own, often bouncing off walls or pursuing the player.

Enemy	Initial Setup	Movement Logic
Yale	go to x = 0, y = 0; point direction 90°	Forever: <i>If touching left wall or right wall</i> → turn 180° (bounce). move 1 step.
MIT	go to random position; point toward Harvard sprite	Forever: point toward Harvard; move 1 step.

- **Difficulty tuning:** Increase the step size (e.g., move 10 steps) to make enemies faster, creating a harder gameplay level.
- Over-large step values can cause visual “twitching” because the sprite overshoots the target and repeatedly corrects its direction.

▢ Game Design Iterations & Difficulty Scaling

1. **Version 0** – Only static sprites; no code.
 2. **Version 1** – Oscar reacts to mouse hover (lid animation).
 3. **Version 2** – Trash sprite gains falling and drag capability; collision teleports trash back to the top.
 4. **Version 3** – Refactored duplicate code into a custom go to top block.
 5. **Version 4** – Score variable added; points awarded on successful collection.
- Each iteration adds a **new layer of abstraction**, illustrating the incremental development approach emphasized in earlier sections on **algorithm design** and **modular programming**.

▢ Design Principles Highlighted

Principle	Illustration in Oscar Game
Abstraction	Custom block go to top hides coordinate math.

Modularity	Separate scripts for movement, collision, and scoring.
Reusability	Same go to top block called by multiple scripts.
Feedback	Real-time score display gives the player immediate reinforcement.
Scalability	Adding more trash sprites, enemies, or levels follows the same pattern without rewriting core logic.

These notes capture the core mechanics, programming constructs, and design thinking behind the Scratch projects discussed, building directly on the foundational concepts previously covered (loops, conditionals, variables, and pseudocode).